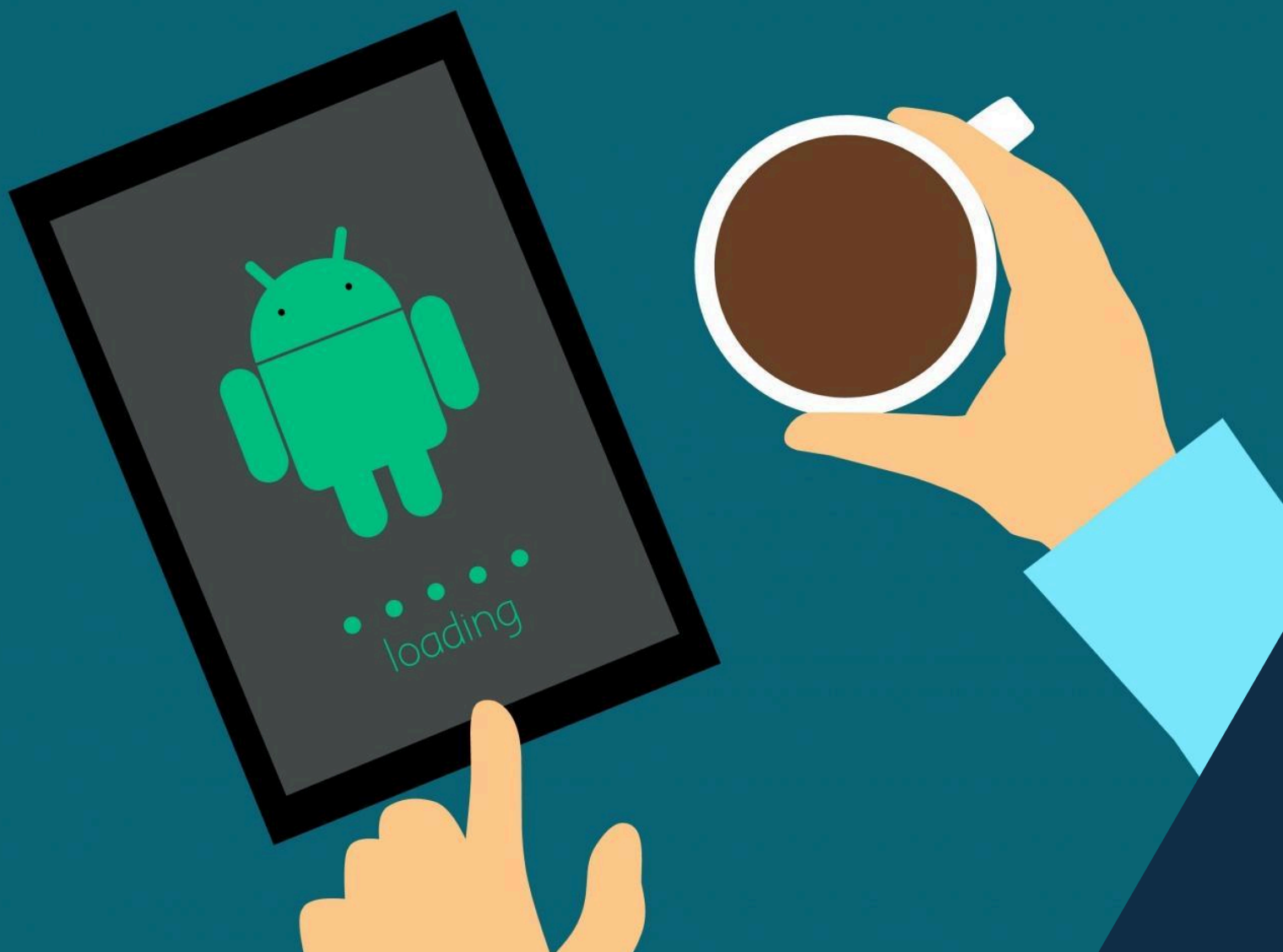




Práctica 1 · Mi primera app Android

Jose Luis Martin Blanco



Índice

Introducción	3
1. Parte A ANÁLISIS DEL DESARROLLO MÓVIL	3
1.1. Ventajas e inconvenientes frente a escritorio y web	3
1.2. Elección tecnológica: Tabla comparativa	4
1.3. Ciclo de vida de una Activity	5
2. PARTE B: ESPECIFICACIÓN DE LA APLICACIÓN	5
2.1. Idea, público objetivo y objetivos medibles	5
2.2. Requisitos funcionales (RF)	6
2.3. Requisitos no funcionales (RNF)	6
2.4. Casos de uso	7
3. PARTE C: ENTORNO DE DESARROLLO	7
3.1. Creación del proyecto	7
3.2. Configuración de dispositivos virtuales (AVD)	8
3.3. Ejecución	8
3.4. Estructura del proyecto	9
4. PARTE D: RECURSOS DE LA APLICACIÓN	9
4.1. Textos e internacionalización	9
4.2. Imágenes y recursos gráficos	9
4.3. Colores y estilos	10
5. CONCLUSIÓN PERSONAL	10



Introducción

En este documento recojo todo el trabajo que he hecho para la primera práctica del módulo, donde el objetivo principal era crear desde cero una app para Android llamada SenderosSevilla, pensada para gestionar rutas de senderismo. En lugar de meterme a programar lógica complicada desde el primer día, me he centrado en lo importante que es entender cómo funciona el desarrollo en Android, montar bien el proyecto en Android Studio y organizar de forma ordenada todos los recursos que va a usar la aplicación. A lo largo de las siguientes páginas explico paso a paso las decisiones que he tomado para que la app quede limpia, organizada y preparada para funcionar sin problemas en diferentes dispositivos.

1.PARTE A ANÁLISIS DEL DESARROLLO MÓVIL

1.1. Ventajas e inconvenientes frente a escritorio y web

Al enfocar el desarrollo hacia dispositivos móviles frente a los entornos de escritorio o web, se aprecian diferencias fundamentales en la arquitectura y la experiencia de usuario:

- **Ventajas:** Permiten una movilidad total, aprovechan de forma directa el hardware del dispositivo y ofrecen una interacción más directa y fluida con el usuario.
- **Inconvenientes:** Nos enfrentamos a muchas pantallas, a la necesidad estricta de optimizar el consumo de batería y a limitaciones de memoria más restrictivas que en un ordenador.

¿Cómo repercute esto directamente en *SenderosSevilla*?

1. **Optimización de la batería:** El uso continuado del GPS para registrar rutas genera un impacto energético elevado, por lo que debo programar el acceso al sensor de manera eficiente.
2. **Gestión de la memoria RAM:** La carga descontrolada de elementos gráficos pesados puede provocar que el sistema operativo cierre la aplicación de forma imprevista.
3. **Conectividad inestable:** Al movernos por entornos naturales o rurales, la cobertura suele perderse; por ello, la app debe contemplar un almacenamiento local robusto.



4. **Adaptabilidad visual:** La diversidad de resoluciones y densidades exige abandonar los tamaños absolutos y trabajar con unidades independientes de densidad.

1.2. Elección tecnológica: Tabla comparativa

Para justificar la elección de la plataforma, he valorado las tres alternativas principales del mercado:

Tipo de Desarrollo	Descripción general	Rendimiento	Acceso al hardware	Coste / Esfuerzo	Ejemplo
Nativo	Código específico para el sistema operativo	Óptimo y máximo.	Total y directo.	Elevado.	Android Studio
Híbrido	Tecnologías web encapsuladas en un contenedor.	Moderado / Bajo.	Restringido	Bajo.	Ionic
Multiplataforma	Un código base común para compilar en múltiples sistemas.	Alto.	Amplio.	Intermedio.	Flutter

Justificación de mi elección: Me he decantado por el desarrollo nativo en Android porque me permite aprovechar al máximo las capacidades del hardware, trabajar directamente con las herramientas oficiales de Google y cumplir con los objetivos formativos del módulo.



1.3. Ciclo de vida de una Activity

A lo largo de la práctica he comprendido que una pantalla en Android no permanece estática, sino que transita por diversos estados (`onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onDestroy`). En el contexto de mi aplicación, esto se refleja en situaciones cotidianas:

- **Recepción de una llamada:** La actividad pierde el primer plano y pasa por `onPause()`, momento que debo aprovechar para pausar temporalmente el registro de la ruta.
- **Rotación del dispositivo:** El sistema destruye y vuelve a recrear la actividad por completo (`onPause` → `onStop` → `onDestroy` → `onCreate...`), lo que me obliga a gestionar la persistencia temporal para no perder los datos introducidos.
- **Pulsación del botón Atrás:** La actividad se desborda de la pila de navegación transitando hacia `onStop()` y `onDestroy()`.

2. PARTE B: ESPECIFICACIÓN DE LA APLICACIÓN

2.1. Idea, público objetivo y objetivos medibles

- **Idea y público:** *SenderosSevilla* está concebida como una utilidad móvil intuitiva dirigida a excursionistas y deportistas aficionados que desean registrar y consultar sus itinerarios al aire libre.
- **Metas cuantificables:**
 1. Permitir al usuario iniciar el registro de una nueva ruta en menos de tres interacciones desde la pantalla principal.
 2. Garantizar que el listado histórico de rutas cargue en pantalla en un tiempo inferior a dos segundos.
 3. Asegurar la compatibilidad de ejecución en el 100% de los dispositivos configurados con Android 7.0 .



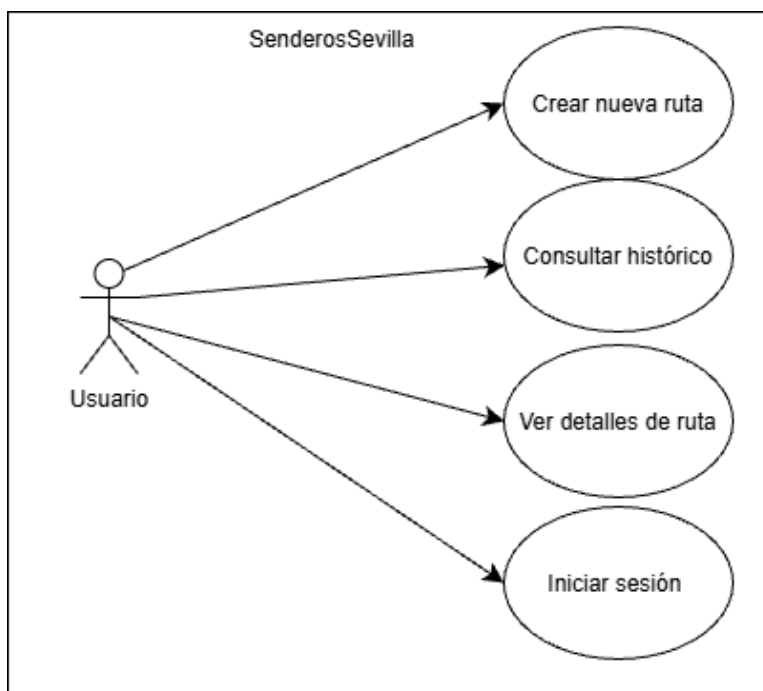
2.2. Requisitos funcionales (RF)

- RF-01: El usuario puede introducir su nombre de perfil al iniciar la aplicación para personalizar la bienvenida. (Prioridad: Alta)
- RF-02: El usuario puede iniciar una nueva ruta especificando el nivel de dificultad del recorrido. (Prioridad: Alta)
- RF-03: El usuario puede visualizar un listado completo con las rutas almacenadas previamente. (Prioridad: Media)
- RF-04: El usuario puede acceder a los detalles técnicos de una ruta seleccionada (distancia y duración). (Prioridad: Media)
- RF-05: El usuario puede modificar las preferencias visuales generales de la interfaz. (Prioridad: Baja)

2.3. Requisitos no funcionales (RNF)

- RNF-01 (Rendimiento) : La interfaz gráfica de la pantalla principal debe completarse en menos de 2 segundos.
- RNF-02 (Compatibilidad) : La aplicación debe ejecutarse de forma nativa desde la versión Android 7.0 (API 24) en adelante.
- RNF-03 (Usabilidad) : Los elementos táctiles interactivos deben mantener un área mínima de 48 dp para facilitar su pulsación.
- RNF-04 (Seguridad) : La información de las rutas debe almacenarse de forma protegida en el directorio interno de la aplicación.

2.4. Casos de uso



- **Ficha del Caso de Uso "Crear ruta":** El usuario accede a la funcionalidad de nueva ruta, selecciona la dificultad y valida la acción. Si algún parámetro obligatorio está ausente, el sistema emite una advertencia visual. Al finalizar, la ruta queda registrada localmente.
- **Ficha del Caso de Uso "Consultar histórico":** El usuario selecciona la opción de historial, el sistema consulta los registros almacenados y despliega la lista en pantalla. Si no existen registros previos, se muestra un aviso informativo.

3. PARTE C: ENTORNO DE DESARROLLO

3.1. Creación del proyecto

He inicializado el proyecto en Android Studio seleccionando la plantilla Empty Views Activity y configurando los siguientes parámetros bajo el lenguaje Java:

- **Nombre del proyecto:** PMDM_P1_JoseLuis.
- **Nombre del paquete:** es.medac.joseluis.app.
- **SDK Mínimo:** API 24: Android 7.0 (Nougat).

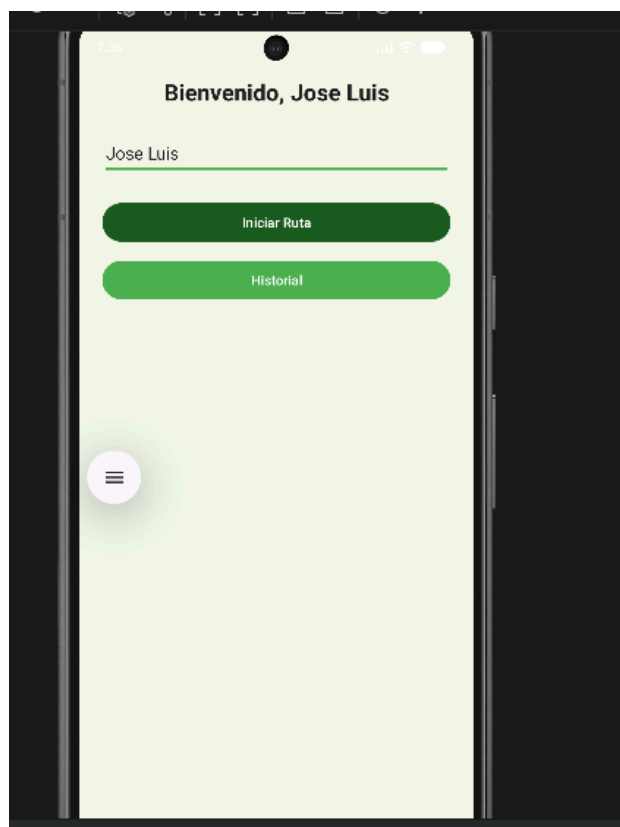


3.2. Configuración de dispositivos virtuales (AVD)

Para comprobar el comportamiento del diseño, he configurado y documentado dos emuladores diferentes en el administrador de dispositivos:

Dispositivo AVD	Modelo	Resolución	Densidad (dpi)	Versión API
Móvil de pruebas	Pixel 7	1080×2400	420 dpi (xxhdpi)	API 34
Tablet de pruebas	Pixel C	2560×1800	320 dpi (xhdpi)	API 34

3.3. Ejecución





3.4. Estructura del proyecto

La arquitectura del proyecto responde a la organización estándar exigida por Android:

- `AndroidManifest.xml`: Archivo de configuración global que declara componentes, actividades, permisos y el icono de la aplicación.
- `java/es.medac.joseluis.app/MainActivity.java`: Directorio del código fuente encargado de la lógica en Java.
- `res/`: Contenedor de recursos estáticos (diseños XML, valores, imágenes y vectores).
- `build.gradle`: Fichero de configuración y control de dependencias y versiones del SDK.
- *Relación*: La conexión entre la lógica y la interfaz se establece explícitamente desde la Activity mediante la llamada `setContentView(R.layout.activity_main)`.

4. PARTE D: RECURSOS DE LA APLICACIÓN

4.1. Textos e internacionalización

He evitado el uso de cadenas de texto embebidas en los ficheros de diseño, externalizando todas las cadenas al archivo `res/values/strings.xml`. He implementado parámetros dinámicos (`%1$s`) y matrices de texto (`string-array`). Así mismo, he habilitado la internacionalización creando el directorio `res/values-en/strings.xml` para la traducción completa al inglés, comprobando su funcionamiento al modificar los parámetros regionales del emulador.

4.2. Imágenes y recursos gráficos

- **Recursos vectoriales (Vector Asset)**: Utilizados para los iconos de la interfaz debido a su capacidad de escala sin pérdida de calidad y su reducido peso de almacenamiento.



- **Mapas de bits:** Reservados para elementos gráficos complejos y optimizados para evitar incrementar innecesariamente el tamaño del archivo APK.
- **Icono adaptativo:** Diseñado mediante la herramienta *Image Asset* de Android Studio, estableciendo capas independientes de fondo y primer plano, y vinculado correctamente en el Manifest.

4.3. Colores y estilos

- **Paleta de colores (`colors.xml`):** He definido una paleta superior a cinco colores empleando nomenclatura semántica orientada a su función (ej. `color_primario`, `color_fondo`), lo que simplifica futuras modificaciones de diseño.
- **Estilos y temas (`themes.xml`):** He configurado estilos reutilizables para estandarizar títulos y botones, aplicando rigurosamente unidades `dp` para dimensiones y `sp` para la escala tipográfica.

5. CONCLUSIÓN PERSONAL

El desarrollo de esta práctica me ha aportado una visión clara y estructurada sobre cómo se inicia un proyecto profesional en Android Studio. Aunque al principio requiere atención la gestión de dependencias en Gradle y la correcta ubicación de los recursos, la separación estricta entre la capa visual (XML) y la lógica (Java) aporta un orden lógico muy eficiente. Considero que he afianzado los conceptos fundamentales para construir aplicaciones ligeras, adaptables a diferentes resoluciones y preparadas para evolucionar técnicamente en fases posteriores del módulo.